

Universitat Politècnica de Catalunya

ESEIAAT

Computational Engineering

Turbulence

Assignment 4 (CFD)

Gerard Martínez Vilella

Contents

1 Burgers Equation in Fourier Space **2**

1.1 Problem Specification 2

1.2 Theoretical Framework 2

1.3 Numerical Methodology 3

1.4 Turbulence Modeling: LES in Fourier Space 3

1.5 Numerical Implementation 3

1.6 Results and Verification 5

1.6.1 Analysis of DNS vs. LES at Re=40 5

1.6.2 Effect of Reynolds Number 6

1.7 Conclusions 8

A Code **10**

A.1 Code Listings 10

A.1.1 Main Script 10

A.1.2 Function: Setup Simulation 10

A.1.3 Function: Run Simulation 11

A.1.4 Function: Load Cached Results 14

A.1.5 Function: Cache Results 14

A.1.6 Function: Get Cache Filename 15

A.1.7 Function: Plot Energy Spectrum 16

1 Burgers Equation in Fourier Space

1.1 Problem Specification

The present study addresses the numerical resolution of the one-dimensional Burgers equation, which serves as a simplified framework for analyzing turbulence phenomena such as the energy cascade, inter-scale interactions, and the balance between convective and diffusive effects.

The governing equation is expressed as:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \frac{1}{Re}\frac{\partial^2 u}{\partial x^2}, \tag{1}$$

where $u(x,t)$ represents the velocity field and Re is the Reynolds number. The analysis focuses on two configurations: a Direct Numerical Simulation (DNS), where all scales are directly resolved, and a Large Eddy Simulation (LES), in which unresolved scales are modeled using a spectral eddy viscosity.

1.2 Theoretical Framework

To efficiently handle derivatives and energy transfer across scales, the Burgers equation is reformulated in Fourier space. The transformed equation for each mode k is:

$$\frac{d\hat{u}_k}{dt} + \sum_{k=p+q} \hat{u}_p(iq)\hat{u}_q = -\nu k^2 \hat{u}_k + F_k, \tag{2}$$

where $\nu = 1/Re$ is the kinematic viscosity, and F_k is an external forcing term. The nonlinear convolution term represents triadic interactions among modes, governing the transfer of energy from large to small scales.

The physical velocity field can be reconstructed from its Fourier coefficients as:

$$u(x,t) = \sum_{k=-N}^{N} \hat{u}_k(t) e^{ikx}. \tag{3}$$

1.3 Numerical Methodology

An explicit Euler scheme is employed for the temporal integration, constrained by a Courant–Friedrichs–Lewy (CFL) stability condition:

$$\Delta t = C_1 \frac{Re}{N^2}, \tag{4}$$

where $C_1 = 0.025$. The solver iterates until the difference between consecutive solutions satisfies a tolerance criterion of 10^{-5}, or until a maximum of 10^5 iterations.

For the energy balance, the temporal evolution of the kinetic energy in each mode is given by:

$$\frac{dE_k}{dt} = -\frac{2k^2}{Re} E_k - \left(\hat{u}_k C_k(\hat{u}_p, \hat{u}_q) + \hat{u}_k^* C_k^*(\hat{u}_p, \hat{u}_q)\right) + \hat{u}_k F_k + F_k \hat{u}_k^*, \tag{5}$$

where $C_k(\hat{u}_p, \hat{u}_q)$ represents the nonlinear convective term.

1.4 Turbulence Modeling: LES in Fourier Space

In the LES configuration, a spectral eddy viscosity model is introduced to account for unresolved scales. The turbulent viscosity is defined as:

$$\nu_t(k) = \nu + \nu_t^\infty \sqrt{\frac{E_k}{E_{k_N}}} \left(1 + 34.5 e^{-3.03(k_N/k)}\right), \tag{6}$$

where E_k denotes the kinetic energy spectrum and k_N is the cutoff frequency. The asymptotic viscosity term is computed as:

$$\nu_t^\infty = \frac{0.315 - m}{m + 1} \sqrt{\frac{3 - m}{C_k}}, \tag{7}$$

with $m = 2$ and $C_k = 0.4523$.

This formulation enhances numerical stability for coarse meshes while preserving energy transfer consistency.

1.5 Numerical Implementation

The simulation is implemented in MATLAB using a modular approach. The main script `main.m` orchestrates the entire process, handling initialization, execution, caching, and visualization.

The script begins with workspace cleanup and loads simulation parameters via `setupSimulation()`,

which configures multiple simulation cases with different parameters (Reynolds number Re, number of modes N, and LES coefficient C_k). The implementation includes a caching system that stores computed results and automatically loads them if the same parameter set has been previously computed, significantly reducing computation time when testing different configurations.

The core computation is performed by `runSimulation(params)`, which iterates through all defined cases. For each case, the solver:

1. Initializes the velocity field in Fourier space with $\hat{u}_k = 1/k$ for modes $k = 1, \ldots, N$, maintaining $\hat{u}_1 = 1$ as a constraint.
2. Computes the convective term through triadic interactions: for each mode j, it sums contributions from all mode pairs (p, q) satisfying $j = p + q$, handling both positive and negative modes through conjugate symmetry.
3. Applies the diffusive term: for DNS cases, standard molecular viscosity $\nu = 1/Re$ is used; for LES cases, the spectral eddy viscosity model is incorporated.
4. Updates the velocity field using an explicit Euler scheme with time step $\Delta t = C_1 Re/N^2$.
5. Iterates until convergence, defined as $\max |\hat{u}_k^{(n+1)} - \hat{u}_k^{(n)}| < 10^{-5}$, or until reaching the maximum iteration limit of 10^5.

After convergence, the kinetic energy spectrum $E_k = |\hat{u}_k|^2$ is computed for each case. The results are automatically cached to disk, and the script generates comparative visualizations via `plotEnergySpectrum(results, params)`, which plots all cases on a logarithmic scale along with a reference slope $E_k \sim k^{-m}$.

The implementation relies on the following key functions:

- `setupSimulation()`: Defines multiple simulation cases with varying parameters (Re, N, C_k, case type). Cases can be easily added, removed, or modified to test different configurations.
- `runSimulation(params)`: Executes the iterative solver for all cases, computing convective and diffusive terms in-place, and returns a structure array containing results for each case.
- `cacheResults()`, `loadCachedResults()`, `getCacheFilename()`: Manage result caching, generating unique filenames based on all simulation parameters to ensure cache validity.

- `plotEnergySpectrum()`: Creates comparative logarithmic plots of energy spectra for all computed cases, distinguishing DNS and LES results with different markers and including a reference theoretical slope.

This modular structure facilitates code maintenance, enables systematic parametric studies by easily modifying cases in `setupSimulation()`, and allows for efficient comparison between DNS and LES configurations across different resolutions and model parameters.

1.6 Results and Verification

The solver was validated by comparing the numerical energy spectra from multiple simulation cases against the theoretical $E_k \sim k^{-2}$ energy decay law. The analysis focuses on the performance of under-resolved DNS, the stabilizing effect of the LES model, and the influence of the Reynolds number.

1.6.1 Analysis of DNS vs. LES at Re=40

The primary test case at $Re = 40$ (Figure 1) clearly illustrates the core problem and the proposed solution.

- **DNS (N=100):** This high-resolution simulation serves as the benchmark or "ground truth." It correctly captures the physics, following the $E_k \sim k^{-2}$ slope for the inertial range before viscosity dominates at high wavenumbers, causing a sharp energy drop-off in the dissipation range.
- **DNS (N=20):** This under-resolved simulation fails catastrophically. While it follows the benchmark at very low k, energy begins to accumulate non-physically near its cutoff wavenumber ($k = 20$). This "blow-up" (the large blue spike) is a classic aliasing instability, as energy that should be transferred to $k > 20$ is folded back, leading to a divergent solution.
- **LES (N=20, $C_k = 0.4523$):** Using the same coarse N=20 grid, the LES model with the theoretical C_k constant (yellow line) performs better. The spectral eddy viscosity acts as an "energy sink" at high wavenumbers, modeling the dissipation that the coarse grid cannot resolve, although not being able to support ¿ 10. This entirely removes the instability and allows the N=20 simulation to more accurately track the DNS N=100 benchmark.
- **LES (N=20, $C_k = 0.05$):** This case (purple line) demonstrates the sensitivity to the model constant. While it also stabilizes the simulation (preventing the blow-up), it is *overly dissipative*. The energy spectrum drops off from the theoretical slope far

too early, indicating that this ad-hoc constant removes too much energy, resulting in an inaccurate physical representation.

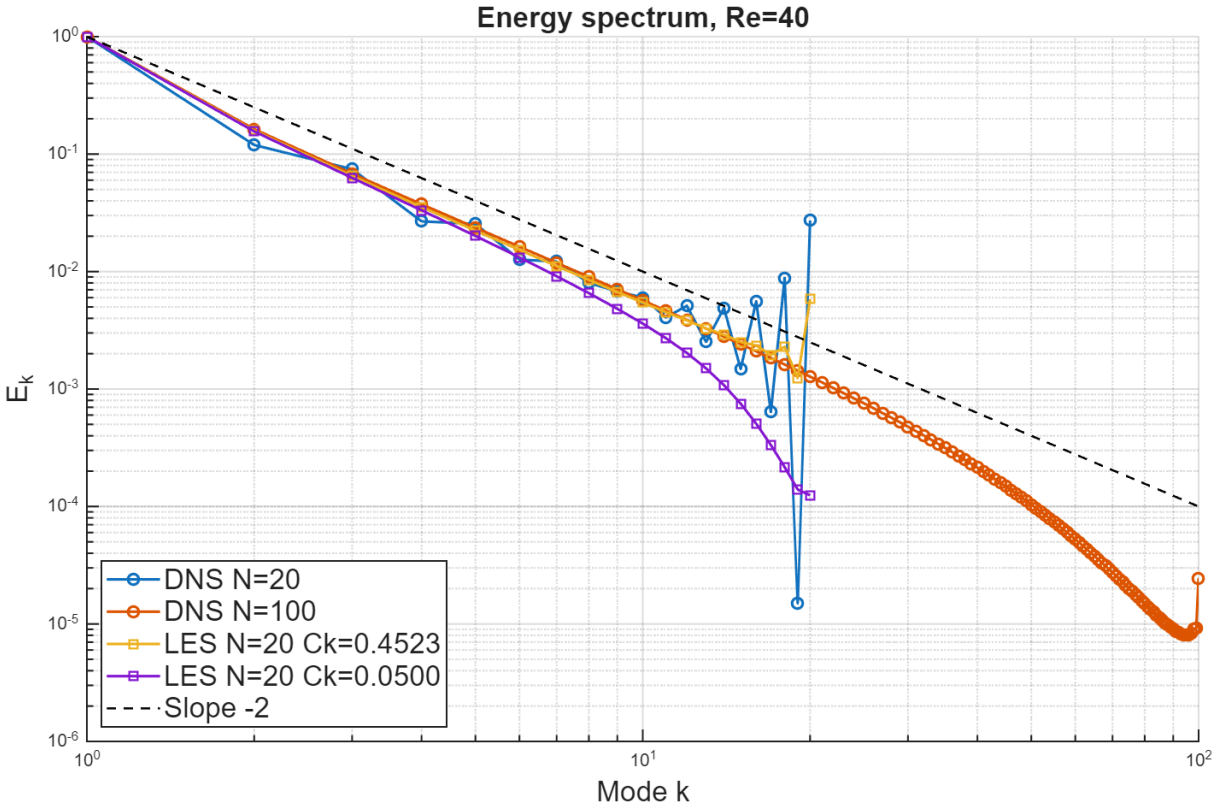


Figure 1: Energy spectrum E_k as a function of mode number k for Re = 40. The reference slope $E_k \sim k^{-2}$ is shown as a dashed line.

1.6.2 Effect of Reynolds Number

Comparing the results at $Re = 20$ (Figure 2) and $Re = 80$ (Figure 3) reveals the critical role of Re in numerical stability. It is important to note that a more accurate time step has been needed for higher Reynolds, going from $C_1 = 0.025$ to $C_1 = 0.01$.

- The high-resolution DNS (N=100) benchmarks behave as expected: at $Re = 20$, physical viscosity is higher, so the dissipation range begins at a lower k (larger scales). At $Re = 80$, the k^{-2} inertial range is wider, and dissipation is confined to the smallest scales (highest k).
- The failure of the under-resolved DNS (N=20) is **strongly dependent on** Re. At $Re = 20$, the instability is present but less severe, as the high physical viscosity ($1/Re$) provides some natural damping. At $Re = 80$, the physical viscosity is low, and the DNS (N=20) simulation blows up far more violently than in the $Re = 40$ case.

- In both cases, the LES (N=20, $C_k = 0.4523$) model successfully stabilizes the simulation and reproduces the correct benchmark spectrum, proving its robustness across different Re numbers.

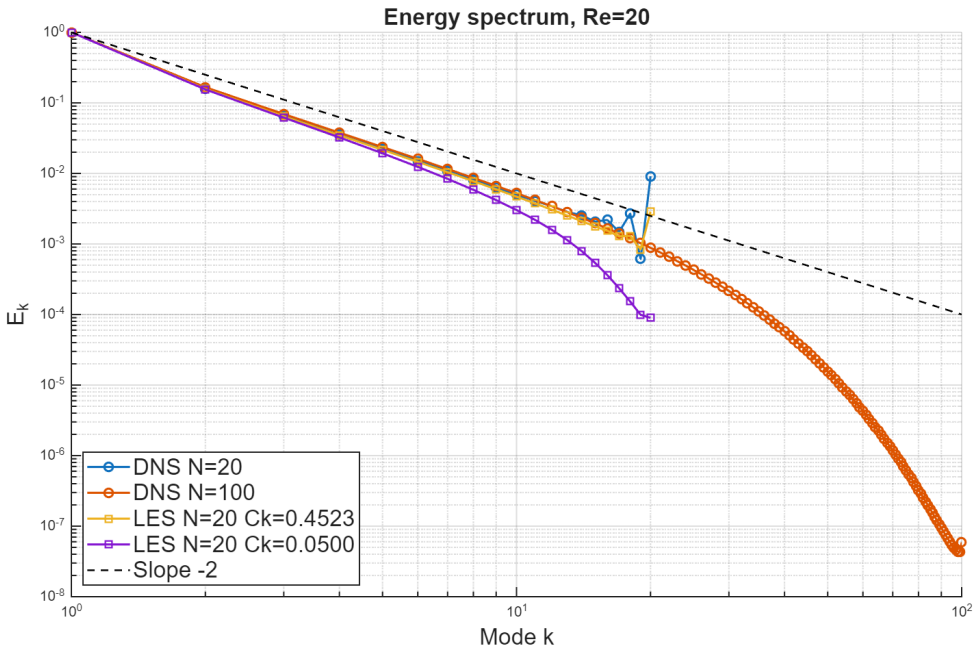


Figure 2: Energy spectrum E_k as a function of mode number k for Re = 20. The reference slope $E_k \sim k^{-2}$ is shown as a dashed line.

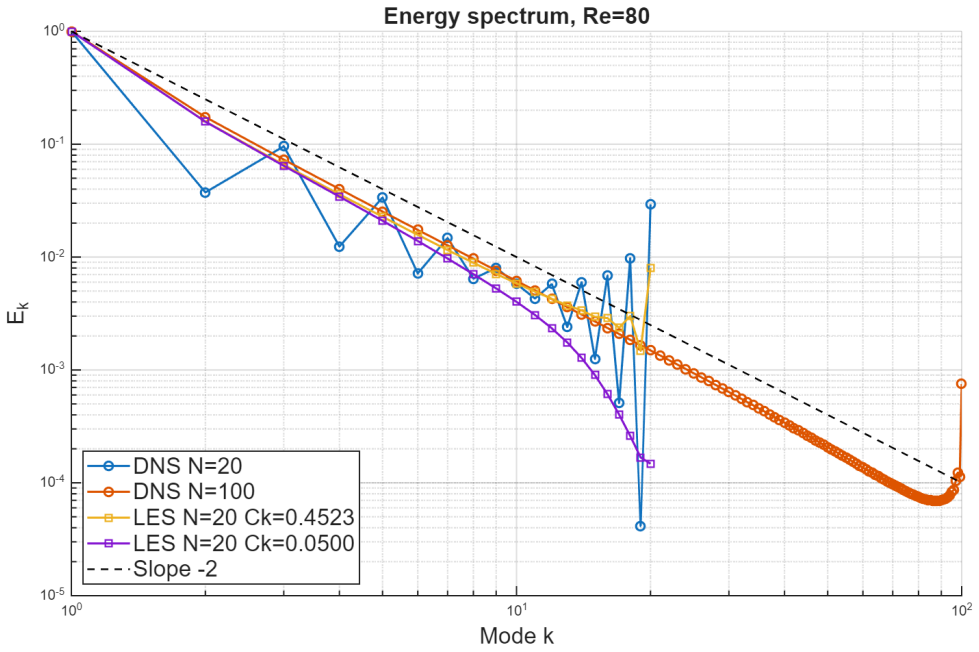


Figure 3: Energy spectrum E_k as a function of mode number k for Re = 80. The reference slope $E_k \sim k^{-2}$ is shown as a dashed line.

Finally, Figure 4 explores the effect of the LES model on an already-resolved N=100 grid. As shown, the LES N=100 cases (light blue and green) lie almost perfectly on top of the DNS N=100 benchmark. This is an important verification: the LES model is "smart" in that it applies minimal artificial viscosity when the grid is fine enough to resolve the dissipation physically, confirming it does not contaminate a well-resolved simulation.

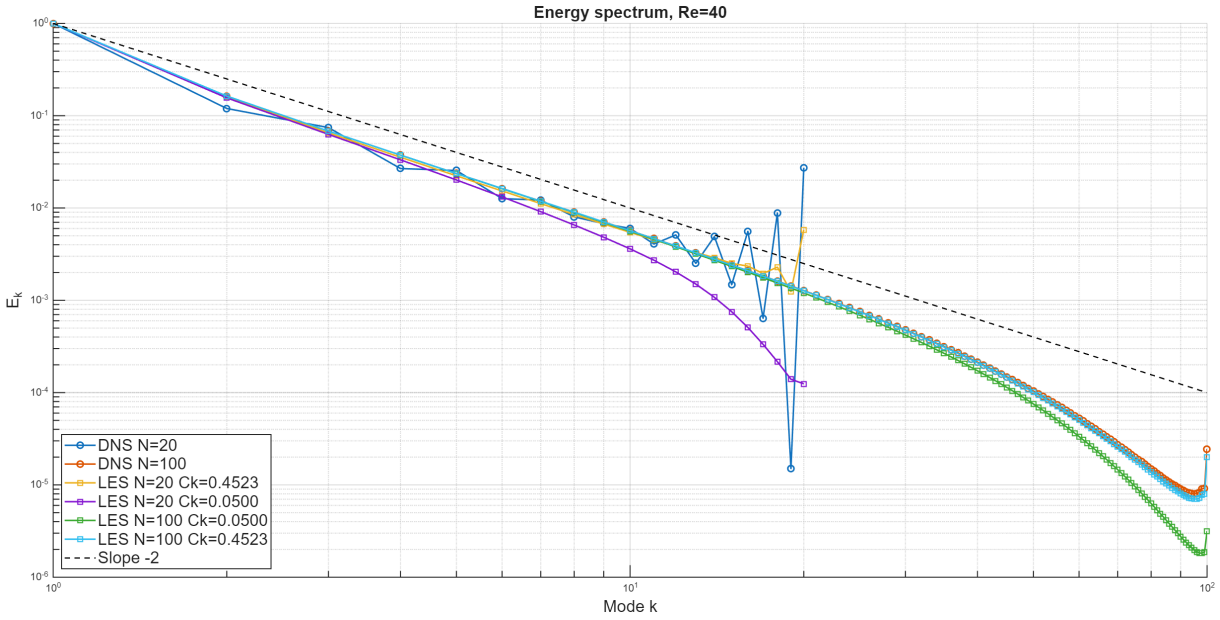


Figure 4: Energy spectrum E_k as a function of mode number k for Re = 40, but adding extra LES N=100 cases. The reference slope $E_k \sim k^{-2}$ is shown as a dashed line.

1.7 Conclusions

The implemented solver successfully reproduces the theoretical dynamics of the Burgers equation in Fourier space, confirming the expected $E_k \sim k^{-2}$ energy decay law in all high-resolution DNS benchmark cases.

The results clearly demonstrate the primary challenge of turbulence simulation: under-resolved DNS (N=20) is numerically unstable, and this instability dramatically worsens as the Reynolds number increases. This confirms that simple grid coarsening is not a viable strategy for simulating high-Re flows.

The spectral eddy-viscosity LES model proved to be an effective solution. It successfully stabilized the coarse (N=20) simulations across all Reynolds numbers, eliminating the numerical blow-up. More importantly, the LES (N=20) simulations with the theoretical constant $C_k = 0.4523$ accurately reproduced the energy spectra of the high-fidelity DNS (N=100) benchmarks, demonstrating the trade-off between computational cost and modeling

accuracy. The analysis also showed that the model's effectiveness is sensitive to the C_k constant, as an ad-hoc value ($C_k = 0.05$) led to an overly dissipative and inaccurate solution.

This study validates the implemented DNS and LES solvers and reinforces the understanding of energy transfer and dissipation in turbulent-like flows. The modular code structure, combined with the caching system, provides an efficient and robust framework for further parametric studies.

A Code

A.1 Code Listings

A.1.1 Main Script

Listing 1: Main script for Burgers equation solver

```
clearvars; close all; clc;
fprintf("Initializing Burgers simulation...\n");
params = setupSimulation();
fprintf("Loaded parameters: Re=%d, cases=%d\n", params.Re, numel(params.
    cases));

% Try to load cached results
fprintf("Checking for cached results...\n");
[results, cached_params, found] = loadCachedResults(params);

if found
    fprintf("Using cached results (computed previously).\n");
    % Use cached params in case they differ slightly
    params = cached_params;
else
    fprintf("No cache found. Running simulations...\n");
    results = runSimulation(params);
    fprintf("Simulation runs completed. Caching results...\n");
    cacheResults(results, params);
end

fprintf("Generating figures...\n");
plotEnergySpectrum(results, params);
fprintf("All outputs generated.\n");
```

A.1.2 Function: Setup Simulation

Listing 2: Function: Setup Simulation

```
function params = setupSimulation()
% Define simulation cases:

params.cases = struct('type', {}, 'N', {}, 'Ck', {});
params.cases(1).type = 'DNS';
params.cases(1).N = 20;
params.cases(1).Ck = [];

params.cases(2).type = 'DNS';
```

```
params.cases(2).N = 100;
params.cases(2).Ck = [];

params.cases(3).type = 'LES';
params.cases(3).N = 20;
params.cases(3).Ck = 0.4523;

params.cases(4).type = 'LES';
params.cases(4).N = 20;
params.cases(4).Ck = 0.05;

% params.cases(5).type = 'LES';
% params.cases(5).N = 100;
% params.cases(5).Ck = 0.05;
%
% params.cases(6).type = 'LES';
% params.cases(6).N = 100;
% params.cases(6).Ck = 0.4523;

params.Re = 80;
params.C1 = 0.025;  % Time step coefficient
params.tolerance = 1e-05;  % Convergence tolerance
params.maxIt = 10^5;  % Maximum iterations
params.m = 2;  % Slope in the energy plot
end
```

A.1.3 Function: Run Simulation

Listing 3: Function: Run Simulation

```
function results = runSimulation(params)
num_cases = numel(params.cases);
results = repmat(struct("type", [], "N", [], "Ck", [], "k", [], "E_k",
    [], "time", [], "energy", []), num_cases, 1);

for idx = 1:num_cases
    case_info = params.cases(idx);
    N = case_info.N;
    case_type = case_info.type;

    fprintf("  Case %d/%d: %s N=%d", idx, num_cases, case_type, N);
    if strcmp(case_type, 'LES')
        fprintf(" Ck=%.4f", case_info.Ck);
    end
    fprintf("\n");

    % Initialize: use positive modes only
```

```
    k = 1:N;  % Mode numbers
    u = 1 ./ k;  % Initial velocities
    time_step = params.C1 * params.Re / (N^2);

    % Iterative solver
    for iter = 1:params.maxIt
        % Preallocate
        conv = zeros(1, N);
        diff = zeros(1, N);

        % Compute convective and diffusive terms for each mode j
        for j = 1:N
            % Convective term
            for p = (-N+j):N
                q = j - p;
                if p ~= 0 && q ~= 0
                    % Handle positive and negative modes
                    % Check bounds to avoid index errors
                    if p > 0 && p <= N
                        u_p = u(p);
                    elseif p < 0 && abs(p) <= N
                        u_p = conj(u(-p));
                    else
                        continue;  % Skip if out of bounds
                    end

                    if q > 0 && q <= N
                        u_q = u(q);
                    elseif q < 0 && abs(q) <= N
                        u_q = conj(u(-q));
                    else
                        continue;  % Skip if out of bounds
                    end

                    conv(j) = conv(j) + u_p * q * 1i * u_q;
                end
            end

            % Diffusive term
            if strcmp(case_type, 'DNS')
                % DNS: standard diffusion
                diff(j) = (k(j)^2) * u(j) / params.Re;
            else
                % LES: with eddy viscosity model
                C_k = case_info.Ck;
                MetLes = 0.31 * (5 - params.m) / (params.m + 1) * sqrt(3
                    - params.m) * C_k^(-3/2);
```

```
                eddy = 1 + 34.5 * exp(-3.03 * (k(N) / k(j)));
                E_k_n = u(N) * conj(u(N));
                nu_eddy = MetLes * (E_k_n / k(N))^0.5 * eddy;
                diff(j) = k(j)^2 * u(j) * (1/params.Re + nu_eddy);
            end
        end

        % Update velocities
        u_after = u + time_step * (conv - diff);
        u_after(1) = 1;  % Maintain first mode

        % Check convergence
        err = abs(u_after - u);
        if max(err(:)) < params.tolerance
            fprintf("    Converged after %d iterations\n", iter);
            break;
        else
            u = u_after;
        end

        if mod(iter, 10000) == 0
            fprintf("    Iteration %d/%d, max error: %.2e\n", iter,
               params.maxIt, max(err(:)));
        end
    end

    if iter == params.maxIt
        fprintf("    Warning: Maximum iterations reached\n");
    end

    % Calculate kinetic energy spectrum
    E_k = real(u_after .* conj(u_after));  % Ensure real values (remove
       tiny numerical errors)

    % Store results
    results(idx).type = case_type;
    results(idx).N = N;
    if strcmp(case_type, 'LES')
        results(idx).Ck = case_info.Ck;
    else
        results(idx).Ck = [];
    end
    results(idx).k = k;
    results(idx).E_k = E_k;
    results(idx).time = iter * time_step;  % Total time to convergence
    results(idx).energy = sum(E_k);

```

```
108     fprintf("  Case %d completed.\n", idx);
109 end
110 end
```

A.1.4 Function: Load Cached Results

Listing 4: Function: Load Cached Results

```
1  function [results, params, found] = loadCachedResults(params, cacheDir)
2  % LOADCACHEDRESULTS Load simulation results from cache if available
3  %   [results, params, found] = loadCachedResults(params, cacheDir)
4  %   attempts to load cached results matching the given parameters.
5  %
6  %   Inputs:
7  %       params   - struct with simulation parameters
8  %       cacheDir - string path to cache directory (default: 'cache')
9  %
10 %   Outputs:
11 %       results - struct array with simulation results (if found)
12 %       params  - struct with simulation parameters (from cache)
13 %       found   - logical, true if cache was found and loaded
14
15 if nargin < 2
16     cacheDir = 'cache';
17 end
18
19 % Generate cache filename
20 cacheFile = getCacheFilename(params, cacheDir);
21
22 % Check if cache exists
23 if exist(cacheFile, 'file')
24     fprintf('  Loading cached results from: %s\n', cacheFile);
25     data = load(cacheFile);
26     results = data.results;
27     params = data.params;
28     found = true;
29 else
30     results = [];
31     found = false;
32 end
33 end
```

A.1.5 Function: Cache Results

Listing 5: Function: Cache Results

```
function cacheResults(results, params, cacheDir)
% CACHERESULTS Save simulation results to cache directory
%   cacheResults(results, params, cacheDir) saves the results structure
%   and params to a cache file based on a hash of the case parameters.
%
%   Inputs:
%       results - struct array with simulation results
%       params  - struct with simulation parameters
%       cacheDir - string path to cache directory (default: 'cache')

if nargin < 3
    cacheDir = 'cache';
end

% Create cache directory if it doesn't exist
if ~exist(cacheDir, 'dir')
    mkdir(cacheDir);
end

% Generate cache filename based on parameters
cacheFile = getCacheFilename(params, cacheDir);

% Save results and params
save(cacheFile, 'results', 'params', '-v7.3');
fprintf('  Results cached to: %s\n', cacheFile);
end
```

A.1.6 Function: Get Cache Filename

Listing 6: Function: Get Cache Filename

```
function filename = getCacheFilename(params, cacheDir)
% GETCACHEFILENAME Generate a unique cache filename based on parameters
%   filename = getCacheFilename(params, cacheDir) creates a filename
%   based on the case parameters to uniquely identify the simulation.
%
%   Inputs:
%       params  - struct with simulation parameters
%       cacheDir - string path to cache directory
%
%   Output:
%       filename - full path to cache file

if nargin < 2
    cacheDir = 'cache';
end
```

```
16
17 % Create a string representation of all cases
18 caseStr = '';
19 for idx = 1:numel(params.cases)
20     case_info = params.cases(idx);
21     if strcmp(case_info.type, 'LES')
22         caseStr = sprintf('%s_%s_N%d_Ck%.6f', caseStr, case_info.type,
               ...
23              case_info.N, case_info.Ck);
24     else
25         caseStr = sprintf('%s_%s_N%d', caseStr, case_info.type,
               case_info.N);
26     end
27 end
28
29 % Include other important parameters
30 paramStr = sprintf('Re%d_C1%.6f_tol%.2e_maxIt%d_m%d', ...
31     params.Re, params.C1, params.tolerance, params.maxIt, params.m);
32
33 % Create hash-like filename (remove leading underscore if present)
34 if ~isempty(caseStr) && caseStr(1) == '_'
35     caseStr = caseStr(2:end);
36 end
37 fullStr = [caseStr '_' paramStr];
38
39 % Replace problematic characters for filename
40 fullStr = regexprep(fullStr, '[\.\+\-\s]', '_');
41 fullStr = regexprep(fullStr, '_+', '_');  % Replace multiple underscores
       with single
42
43 filename = fullfile(cacheDir, [fullStr '.mat']);
44 end
```

A.1.7 Function: Plot Energy Spectrum

Listing 7: Function: Plot Energy Spectrum

```
1  function plotEnergySpectrum(results, params)
2  fprintf("  Plotting energy spectrum (%d cases)...\n", numel(results));
3  figure
4  hold on
5  set(gca, "XScale", "log", "YScale", "log")
6  set(gca, "LineWidth", 1)
7
8  % Plot each case
9  for idx = 1:numel(results)
10     case_info = results(idx);
```

```
    if strcmp(case_info.type, 'DNS')
        loglog(case_info.k, case_info.E_k, "o-", "LineWidth", 1.5, ...
            "DisplayName", sprintf("DNS N=%d", case_info.N))
    else
        loglog(case_info.k, case_info.E_k, "s-", "LineWidth", 1.5, ...
            "DisplayName", sprintf("LES N=%d Ck=%.4f", case_info.N,
                case_info.Ck))
    end
end

% Add reference slope - use the case with maximum N to span full range
max_N = 0;
max_N_idx = 1;
for idx = 1:numel(results)
    if results(idx).N > max_N
        max_N = results(idx).N;
        max_N_idx = idx;
    end
end

ref_k = results(1).k(1);
ref_val = results(1).E_k(1);
k_ref = results(max_N_idx).k;  % Use k values from case with maximum N
ref_curve = (k_ref / ref_k) .^ (-params.m) * ref_val;
loglog(k_ref, ref_curve, "k--", "LineWidth", 1.2, ...
    "DisplayName", sprintf("Slope -%d", params.m))

xlabel("Mode k")
ylabel("E_k")
title(sprintf("Energy spectrum, Re=%d", params.Re))
legend("Location", "southwest")
grid on
hold off
fprintf("  Energy spectrum plot complete.\n");
end
```